Semi-supervised physics-informed learning for lithium-ion battery state-of-health estimation with scarce capacity measurements: a leakage-free benchmark

Quoc Thang Huynh
Faculty of Computer Science and Engineering,
Ho Chi Minh City University of Technology (HCMUT),
Vietnam National University Ho Chi Minh City, Vietnam

Draft, October 30, 2026

Abstract

Estimating the state of health (SOH) of lithium-ion cells from charging data is a regression problem whose labels are expensive: each label is a full reference discharge. We study how far physics-informed neural networks (PINNs) can reduce that cost, and we do so under an evaluation protocol that avoids the leakage common in this literature. The model is a semi-supervised PINN whose solution network predicts SOH from 16 charge-curve features and is tied to a grey-box degradation rate that is non-negative by construction; its three physics losses do not use labels, so they also act on cells whose capacity was never measured. On four public datasets (387 cells, three chemistries), with cell-wise stratified cross-validation, causal outlier filtering and training-only normalisation, the semi-supervised PINN has 0.39–0.90 times the per-cell mean absolute error (MAE) of an identical network trained on labels alone when 10 % of the cells are labelled, but 0.85–1.09 times at 70 %, and a well-tuned gradient-boosted tree (GBDT) remains the strongest single model. Averaging three PINNs with a GBDT and smoothing the result with a causal trend filter gives an estimator that is better than GBDT in 10 of 16 dataset–label settings, and better than CNN, LSTM and Transformer baselines in 47 of 48 comparisons. A leakage audit shows that row-wise splits make the error of the same model look 1.2–1.8 times smaller, and that per-cell min–max normalisation over the whole life changes it by a factor of 0.46–2.76, depending on the dataset. We re-run the original code of PINN4SOH, a published PINN benchmark on the same datasets, on its own splits and find that its published runs used a monotonicity term computed from labels only, which contributes no gradient; under identical splits and preprocessing our hybrid estimator has 0.76–0.92 times the error of the re-run original code on the full datasets and 0.31–0.67 times in its few-shot settings. Finally, counting labels as capacity measurements rather than cells, measuring every training cell at 1.2–6.0 % of its cycles gives 0.85–0.93 times the error of fully labelling 30 % of the cells, and a few measurements of the cell in service, combined through a learned update model, lower the error further. Code, splits and run configurations are released.

Keywords: lithium-ion battery; state of health; physics-informed neural network; semi-supervised learning; data leakage; cross-validation; benchmark reproducibility; measurement budget.

1 Introduction

The state of health of a lithium-ion cell, its present capacity relative to the nominal one, decides when a cell is derated, repurposed or replaced. Capacity is not observed during normal use; it is measured by a full discharge under reference conditions, which takes hours and interrupts operation. Battery management systems therefore estimate SOH from signals that are always

available, most often from the shape of the charging curve, and learn the mapping from cells whose capacity was measured [1–3]. The cost of such a model is set by how many measured cells, and how many measurements per cell, it needs.

Physics-informed neural networks [4, 5] offer one way to need fewer labels: a loss that encodes known behaviour, such as capacity not recovering on its own, can be evaluated on data without labels. For batteries, PINNs have been used with empirical degradation laws and with learned dynamics [6–8]. Whether the physics actually saves labels, and against which baselines, is less clear. Two issues make published numbers hard to compare. First, the evaluation protocol: consecutive cycles of a cell are almost identical, so a random split by cycle turns the task into interpolation along a trajectory whose labels are known, and normalising each cell by statistics of its whole life gives the model the fraction of life already elapsed, which is not known in service [9]. Second, baselines: gradient-boosted trees are strong on tabular features [10], yet comparisons often include only neural networks.

This paper reports a systematic study under a protocol designed to remove both problems. Our contributions are:

1. A leakage-free protocol for SOH estimation (cell-wise, stratified five-fold cross-validation repeated with different shuffles; causal outlier filtering; normalisation fitted on training cells only; paired per-cell statistics), and an audit that quantifies four common leaks on the same model.

2. A semi-supervised PINN with a grey-box, non-negative degradation rate whose physics losses are label-free, and a comparison with nine baselines at 10–70 % labelled cells on four datasets.

3. A hybrid estimator (three PINNs, a GBDT and a causal trend filter) chosen on validation data only.

4. A re-examination of the PINN4SOH benchmark [6]: we re-run its released code on its own splits, identify that the published configuration's monotonicity term carried no gradient, and compare under identical splits and preprocessing.

5. An analysis of the measurement budget, in which labels are counted as capacity measurements: sparse measurements on all cells versus full labels on a subset, and updating the estimate of a cell in service from a few measurements.

2 Related work

Data-driven SOH estimation maps features of charge or discharge curves, impedance or relaxation voltage to capacity [1, 11, 12]. Gaussian processes [13] and tree ensembles are common for engineered features; recurrent, convolutional and attention models [14–17] operate on sequences of cycles. Physics enters either through the model structure, e.g. equivalent circuits and electrochemical models [18, 19], or through the loss. PINN4SOH [6] couples a solution network with a learned dynamical model $u_t = G(\mathbf{x}, t, u, u_{\mathbf{x}}, u_t)$ and reports results on the four datasets used here. Wen et al. [7] build PINNs around empirical degradation models, and physics-informed approaches to degradation diagnostics are compared by Navidi et al. [8] and discussed by Aykol et al. [20]. Semi-supervised SOH estimation without additional degradation experiments has been studied by Lu et al. [21]. Leakage in machine-learning-based science is documented broadly [9]; for batteries, Roman et al. [1] stress cell-level splits and careful pipelines. Our work combines these threads in one controlled comparison and adds an explicit re-run of a published benchmark.

Table 1: Datasets after preprocessing. Protocols: distinct test conditions used for stratification. Life: number of recorded cycles (min / median / max).

Dataset	Chemistry	Cells	Cycles	Protocols	Life	Initial SOH	Final SOH	Q_{nom} (Ah)
XJTU	NCM	55	22,431	6	110 / 392 / 942	0.83–1.00	0.78–0.89	2.0
TJU	NCA, NCM, NCM+NCA	130	56,697	9	25 / 377 / 1,246	0.88–0.98	0.66–0.90	3.5/2.5
MIT	LFP	125	82,450	3	36 / 622 / 2,156	0.95–1.00	0.82–0.97	1.1
HUST	LFP	77	144,224	1	1,123 / 1,855 / 2,672	1.06–1.12	0.80–0.82	1.1
Total		387	305,802					

3 Data and evaluation protocol

3.1 Datasets and features

We use four public cycling datasets, all as released with PINN4SOH [6], which share one feature extraction (Table 1): XJTU (55 NCM cells, six protocols), TJU [12] (130 NCA, NCM and NCM+NCA cells at 25, 35 and 45 °C), MIT [11] (125 LFP cells, fast-charging protocols) and HUST [22] (77 LFP cells, multi-step discharge). Each cycle is described by 16 statistics of the constant-current voltage and constant-voltage current segments of the charge (mean, standard deviation, kurtosis, skewness, slope, entropy, charge and duration). The label is $y_{c,n} = Q_{c,n}/Q_{\mathrm{nom}}$. The cycle index enters as $t_n = n/1000$ with one constant for all cells; scaling by the life of each cell would reveal the future.

3.2 Preprocessing without future information

A cycle is discarded if any feature is not finite or deviates from the previous 50 cycles of the same cell by more than five robust standard deviations (median and interquartile range of that window, at least ten cycles of history). The threshold removes 1.6 % of cycles, comparable to the 2.2 % removed by a whole-life 3σ filter, but uses only the past. Features are z-scored with means and standard deviations of the training cells, labelled and unlabelled; validation and test cells never enter the normaliser.

3.3 Cell-wise stratified cross-validation

All cycles of a cell belong to the same part. With random row-wise splits, a test cycle is on average $\mathbb{E}[d] = 1/(1-(1-q)^2)$ cycles from the nearest training cycle when a fraction q of rows is used for training (1.10 cycles at $q = 0.7$), so copying the label of the nearest training cycle has an error of at most $\mathbb{E}[d]$ times the mean absolute change between consecutive cycles. We use $K = 5$ folds stratified by test protocol: cells are shuffled within protocols and assigned to folds by a counter that runs across protocols, so every fold contains every protocol, every cell is tested exactly once per repetition, and fold sizes differ by at most one cell. Of the remaining cells, round$(0.1N)$ are used for validation (early stopping and all selection) and the rest, 70 % of the cells, for training. A label fraction f marks round(fN) training cells as labelled, following a fixed order, so the labelled sets are nested in f; $f = 0.7$ uses all training cells. The other training cells keep their features and serve as unlabelled data. The shuffle is repeated five times (three for the later experiments), and in fold k of repetition r every model uses the same seed $100r + k$, the same cells and the same normaliser, so comparisons are paired by cell.

3.4 Metric and statistics

The primary metric is the per-cell MAE, $\mathrm{MAE}_{\mathrm{cell}} = |\mathcal{C}|^{-1} \sum_c n_c^{-1} \sum_n |\hat{y}_{c,n} - y_{c,n}|$, which gives each cell one vote; we report it in units of 10^{-3} SOH. Each cell's error is averaged over repetitions before testing. Differences are tested with the two-sided Wilcoxon signed-rank test on per-cell differences [23], with Holm correction across the four datasets [24]; ratios of mean errors come

with 95 % percentile bootstrap intervals resampling cells. Because training sets of folds overlap, we also report where conclusions survive the corrected resampled t-test of Nadeau and Bengio [25].

4 Methods

4.1 Semi-supervised PINN with grey-box kinetics

The solution network $F_\varphi : \mathbb{R}^{17} \to \mathbb{R}$ (SiLU, 17–128–128–64–1) maps standardised features and time to SOH, $u = F_\varphi(\tilde{\mathbf{x}}, t)$. Degradation follows

$$\frac{\partial u}{\partial t} = -\, r(\tilde{\mathbf{x}}, u, T), \qquad r = \operatorname{softplus}\left(G_\theta(\tilde{\mathbf{x}})\right) e^{\lambda \operatorname{clip}(1-u,\,-0.5,\,1)}\, e^{-\frac{E_a}{R_g}\left(\frac{1}{T}-\frac{1}{T_{\mathrm{ref}}}\right)}, \tag{1}$$

with a small rate network G_θ (16–32–32–1), a knee coefficient $\lambda = \operatorname{softplus}(\lambda_{\mathrm{raw}})$ and an Arrhenius term with $E_a = 10^4 \operatorname{softplus}(e_{\mathrm{raw}})$ J/mol. Every factor is positive, so $r \geq 0$ for any parameters and every solution of (1) is non-increasing in t. For pairs of cycles (n_1, n_2) of one cell with $n_2 - n_1 \sim \mathcal{U}\{5, \ldots, 50\}$ and $u_i = F_\varphi(\tilde{\mathbf{x}}_{n_i}, t_{n_i})$,

$$\mathcal{L}_{\mathrm{data}} = \overline{(u-y)^2} \quad \text{(labelled cycles only)}, \qquad \mathcal{L}_{\mathrm{ode}} = \overline{\left(\partial_t u_1 + r(\tilde{\mathbf{x}}_{n_1}, u_1, T)\right)^2}, \tag{2}$$

$$\mathcal{L}_{\mathrm{mono}} = \overline{\operatorname{ReLU}(u_2 - u_1 - \varepsilon)}, \qquad \mathcal{L}_{\mathrm{range}} = \overline{\operatorname{ReLU}(u_1 - 1.15) + \operatorname{ReLU}(0.4 - u_1)}, \tag{3}$$

where the bar denotes the mean over a batch, $\varepsilon = 0.002$ and $\partial_t u$ comes from automatic differentiation; at the first cycle of a cell, $\mathcal{L}_{\mathrm{range}}$ uses the tighter bounds $[0.85, 1.10]$. The loss is $\mathcal{L} = \mathcal{L}_{\mathrm{data}} + w(s)(\alpha\mathcal{L}_{\mathrm{ode}} + \beta\mathcal{L}_{\mathrm{mono}} + \gamma\mathcal{L}_{\mathrm{range}})$ with $\alpha = 0.02$, $\beta = 0.5$, $\gamma = 1$ and a linear warm-up $w(s) = \min(1, s/800)$. None of the three physics losses contains y, so they are evaluated on pairs drawn half from labelled and half from unlabelled training cells (PINN-semi). PINN-sup applies them to labelled cells only.

The physics here is phenomenological: capacity does not recover by itself, ageing accelerates as SOH falls, side reactions follow Arrhenius kinetics, and SOH stays in a plausible range. It shapes the learned function; at inference only F_φ is evaluated (Figure 1).

4.2 Hybrid estimator

The residual error of single models is dominated by a level offset per cell and slow drift (Section 5.3), which suggests averaging models whose per-cell offsets differ. The hybrid estimator, called PINN-hybrid below, is

$$\hat{y}_n = \omega\, \tfrac{1}{3} \textstyle\sum_{m=1}^{3} F_{\varphi_m}(\tilde{\mathbf{x}}_n, t_n) + (1-\omega)\, g(\tilde{\mathbf{x}}_n, t_n), \qquad \tilde{y}_n = \bar{y} + \min(0, \hat{\beta})(t_n - \bar{t}), \tag{4}$$

where F_{φ_m} are three PINN-semi networks that differ only in their seed, g is the GBDT baseline trained on the same labelled cells, and the second step fits a line with non-positive slope to the last w predictions of the cell and evaluates it at t_n (a causal trend filter that does not use labels). The configuration $\omega = \frac{1}{2}$, $w = 10$ was fixed on validation data at repetition 0 with 10 and 50 % labels before any test prediction was read; candidates that were rejected on validation include distillation from the GBDT, a stronger monotonicity weight, an Euler residual along the trajectory and features relative to the first cycles. A tuned variant selects (ω, w) from $\{0.3, \ldots, 0.7\} \times \{5, 10, 20, 40\}$ on the validation cells of each dataset and label fraction.

4.3 Baselines

All models use the same inputs $[\tilde{\mathbf{x}}, t]$, splits, labelled cells and normaliser: ridge regression; a Gaussian process with RBF and white-noise kernels on 1 500 labelled points [26]; a histogram

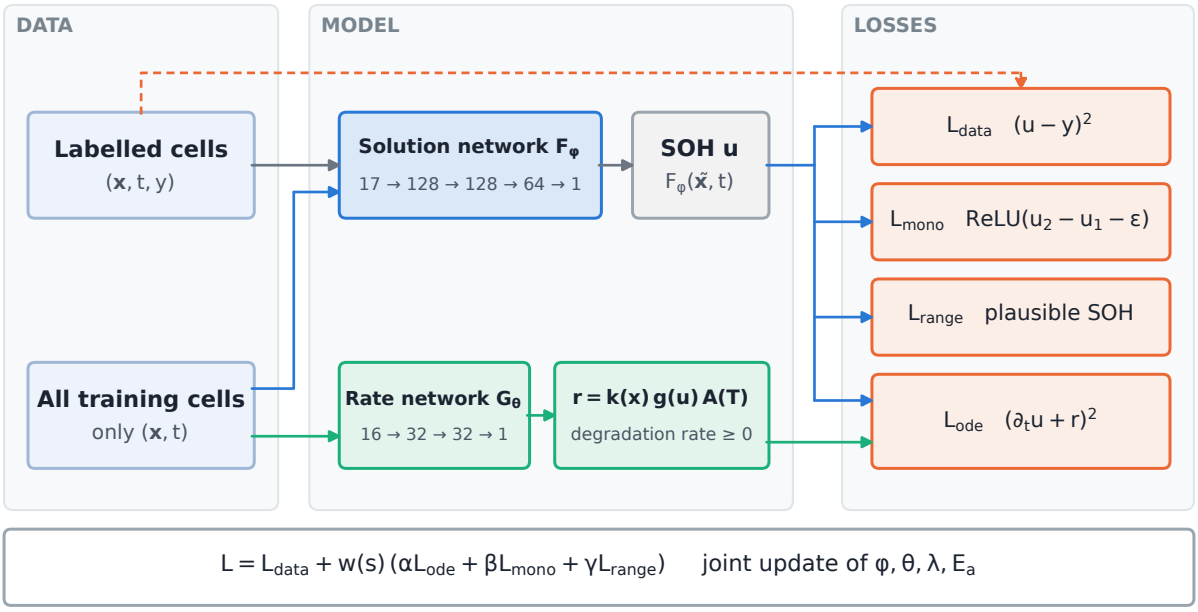


Figure 1: Semi-supervised PINN. The solution network predicts SOH from charge features and time; the rate network and the knee and Arrhenius factors define a non-negative degradation rate. The data loss uses labelled cells only; the monotonicity, range and kinetics losses use pairs of cycles from all training cells, labelled or not.

GBDT [27, 28] with the number of trees chosen on validation; an MLP with the architecture of F_φ trained on $\mathcal{L}_{\text{data}}$; causal CNN, LSTM and Transformer models over the last L cycles of the cell ($L = 16, 48, 16$, chosen on validation); PINN-sup; and a black-box PINN with PINN4SOH-type dynamics $u_t = G(\mathbf{x}, t, u, u_{\mathbf{x}}, u_t)$ and the same label-free losses. Networks are trained with AdamW [29] ($2 \cdot 10^{-3}$, cosine decay over 4 000 steps), batches of 512 labelled points and 512 cycle pairs, and early stopping on validation MAE. Physics weights were chosen once on validation in a preliminary experiment and kept fixed; no hyperparameter was chosen on test data.

4.4 Measurement budget and updates in service

Sparse labels. Instead of labelling every cycle of some cells, every training cell is measured at its first cycle, every qL cycles (L is the median life of the training cells, $q \in \{0.5, 0.2, 0.1, 0.05\}$) and at its last cycle. PINN-semi then uses $\mathcal{L}_{\text{data}}$ on measured cycles only while the physics losses still act on all cycles; the GBDT learns linearly interpolated labels.

Kalman update. When the cell in service is also measured occasionally, the error of the estimate p_n is modelled as $y_n - p_n = b_n + \varepsilon_n$: a cell-specific offset that drifts back to its mean as an Ornstein–Uhlenbeck process, $b_{n'} - \mu_0 = e^{-(n'-n)/\ell}(b_n - \mu_0) + \eta$, plus fast noise with $\operatorname{Var}\varepsilon = R$. A Kalman filter updates b at each measurement and only uses measurements up to the current cycle; μ_0, R and the stationary variance are estimated on the validation cells of the fold and ℓ is chosen on a grid per measurement schedule.

Learned update. The filter only corrects the offset of a model trained without measurements of the cell. The learned update model instead predicts the change since the last measurement m, $\hat{y}_n = y_m + h(\mathbf{u}_n)$, where h is a GBDT on the current features, the change of features since m, y_m, the number of cycles since m in units of L, and the slope between the last two measurements. It is trained on pairs drawn from the labelled training cells under the test-time measurement schedules, with random phases.

Table 2: Leakage audit: per-cell MAE of the same MLP (70 % labels, two repetitions × five folds) under the leakage-free protocol, and the ratio of the MAE under each faulty protocol to it (below 1: the error looks smaller than it is). Last column: copying the label of the nearest training cycle of the same cell under the row-wise split, without any feature.

Dataset	MAE ($\times 10^{-3}$)	row-wise split	per-cell min–max	whole-life 3σ filter	normaliser incl. test	copy nearest
XJTU	13.11	0.55	0.80	0.87	1.03	0.14
TJU	9.86	0.85	1.51	0.98	1.01	0.10
MIT	2.80	0.81	2.76	1.01	1.01	0.15
HUST	17.46	0.64	0.46	0.96	0.98	0.01

5 Results

5.1 Leakage audit

The same MLP at 70 % labels was trained under four protocols that each change one element of the leakage-free protocol (Table 2). A row-wise split lowers the error by a factor of 1.2–1.8, and copying the label of the nearest training cycle, without using any feature, gives 0.01–0.15 times the error of the model evaluated correctly, as the bound of Section 3 predicts. Per-cell min–max normalisation over the whole life, as in the PINN4SOH code, lowers the error on XJTU (0.80) and HUST (0.46), where all cells end at almost the same SOH and the normalised cycle index, the fraction of life already elapsed, nearly determines SOH; it raises the error on TJU (1.51) and MIT (2.76), where end-of-life SOH varies and the normalisation erases the absolute level of the features. Either way the number does not describe deployment, where the life of a cell is unknown. A whole-life outlier filter and a normaliser fitted with test cells have smaller effects, but are leaks as well. All further results use the leakage-free protocol.

5.2 Label efficiency of the semi-supervised PINN

Table 3 gives the per-cell MAE of all single models at four label fractions. With 10 % labelled cells (6–13 labelled cells per fold), PINN-semi has 0.60 (XJTU), 0.90 (TJU), 0.39 (MIT) and 0.61 (HUST) times the error of the MLP, significantly lower on all datasets; at 30 % the ratio is 0.76–0.96, still significant on all four but robust to the conservative fold-level test only on MIT. With 70 % labels, when no unlabelled cell is left and PINN-semi equals PINN-sup, the ratio is 0.85–1.09; on TJU the MLP is significantly better. The geometric mean of the ratio over datasets is 0.60, 0.86, 0.92 and 0.96 at 10, 30, 50 and 70 % labels (Figure 2). Expressed as labels saved, PINN-semi with 10 % labels matches an MLP trained on 17–45 % of the cells.

The GBDT is the strongest single model: it is better than PINN-semi in 14 of 16 settings, 10 of them significantly; the reverse holds significantly only on HUST at 30 % labels. A Gaussian process is also lower than PINN-semi in 10 of 16 settings. The accurate statement is therefore that physics substantially improves the neural network when labels are scarce, bringing it to within 0.95–1.10 times the GBDT at 10 % labels, but does not beat a well-tuned tree on most datasets.

Where the gain comes from. Table 4 separates the components at 30 % labels. Applying the physics losses to the unlabelled cells accounts for 50–64 % of the log-scale reduction relative to the MLP at 10 % labels; the rest comes from physics on labelled cells. At 30 % labels, the monotonicity loss alone provides 62–95 % of the reduction. The black-box dynamics of PINN4SOH perform almost exactly like monotonicity alone (within 2 %): when the dynamics network receives u_t as input it can learn $G \approx u_t$ and satisfy the residual trivially, an escape that the non-negative grey-box rate does not have. Measured on trained models at 30 % labels, MLP predictions increase between cycles at least ten apart in 13.6–31.1 % of pairs and PINN-semi predictions in 3.0–12.0 %, close to the 0.1–7.7 % of the measured labels. Most of the predicted

Table 3: Per-cell MAE ($\times 10^{-3}$) on test cells at four fractions of labelled cells; five repetitions × five folds, every cell tested five times. Bold: lowest per dataset and column.

Dataset	Model	10 %	30 %	50 %	70 %
XJTU	Ridge	18.22	15.54	14.81	14.09
	GPR	14.85	12.11	14.86	19.18
	GBDT	**14.49**	**10.85**	**10.19**	**9.51**
	MLP	24.72	15.95	14.58	12.74
	PINN-sup	20.25	14.54	13.21	12.19
	Black-box PINN	15.68	14.15	12.82	12.34
	PINN-semi	14.93	13.44	13.52	12.19
TJU	Ridge	23.01	21.60	20.50	20.74
	GPR	13.50	9.22	8.01	8.17
	GBDT	**12.68**	**8.14**	**7.21**	**6.77**
	MLP	15.50	11.33	9.99	9.64
	PINN-sup	14.73	10.67	10.20	10.52
	Black-box PINN	13.90	10.91	10.36	10.16
	PINN-semi	13.91	10.85	10.46	10.52
MIT	Ridge	3.53	3.24	3.17	3.13
	GPR	3.21	2.14	1.84	1.79
	GBDT	**3.09**	**1.99**	**1.73**	**1.58**
	MLP	8.54	3.40	3.04	2.86
	PINN-sup	5.34	2.69	2.54	2.44
	Black-box PINN	4.19	2.80	2.61	2.54
	PINN-semi	3.34	2.58	2.48	2.44
HUST	Ridge	**22.88**	18.85	18.32	18.10
	GPR	28.82	21.24	18.50	17.22
	GBDT	24.13	18.63	**15.65**	**14.45**
	MLP	37.95	19.80	17.73	16.94
	PINN-sup	31.67	17.82	16.11	15.99
	Black-box PINN	24.04	17.49	16.47	15.99
	PINN-semi	22.98	**17.30**	16.11	15.99

decline passes through the charge features rather than through t. The learned λ and E_a stay at their initial values; on TJU, the only dataset with several temperatures, raising the ODE weight fifty-fold or changing the initialisation moves them by exactly the change of the initial value. With the rate written as $k(\mathbf{x})g(u)A(T)$, the feature-dependent factor k absorbs any change of λ or E_a, so these parameters are structural, not identified.

5.3 Hybrid estimator and sequence models

At 30 % labels, the errors of the MLP, PINN-semi and GBDT are dominated by per-cell level offsets (14–58 % of the MSE) and slow drift (32–57 %); fast cycle-to-cycle noise accounts for 1–17 % outside MIT. The per-cell offsets of the PINN ensemble and the GBDT are only moderately correlated (Pearson 0.50–0.80) and have opposite signs for 16–31 % of the cells, which is why averaging them helps. With the fixed configuration, PINN-hybrid has a lower MAE than PINN-semi in 16 of 16 settings (all significant; ratio 0.67–0.95) and than the GBDT in 10 of 16 (8 significant), including all four datasets at 10 % labels; the GBDT remains significantly better only on TJU at 70 %. On the two label fractions used during development, the test ratio to PINN-semi (0.805) is close to the validation ratio (0.823), so selection on validation did not make the test result optimistic. The geometric-mean ratio to PINN-semi falls to 0.961 with the ensemble, 0.798 with the GBDT and 0.793 with the filter; most of the gain comes from combining the two model families. With MLPs in place of the PINNs (same GBDT and filter), the physics

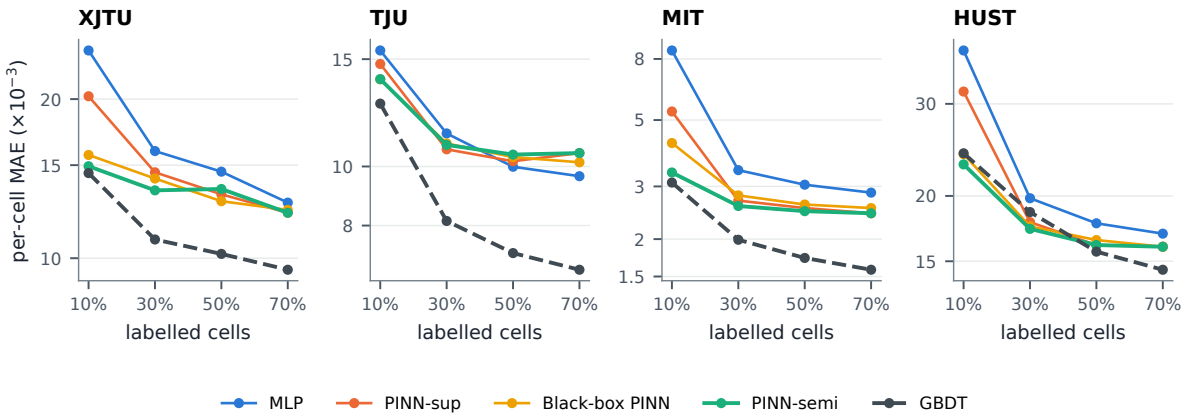


Figure 2: Per-cell MAE against the fraction of labelled cells (log scale). The semi-supervised PINN closes most of the gap between the MLP and the GBDT at 10 % labels; with more labels, the GBDT stays lower on XJTU, TJU and MIT.

Table 4: Source of the gain at 30 % labelled cells: per-cell MAE ($\times 10^{-3}$) and, in parentheses, the ratio to the MLP.

Model (30 % labels)	XJTU	TJU	MIT	HUST
MLP	15.95	11.33	3.40	19.80
PINN-sup (physics on labelled cells only)	14.54 (0.91)	10.67 (0.94)	2.69 (0.79)	17.82 (0.90)
PINN-semi, monotonicity only ($\alpha = \gamma = 0$)	14.13 (0.89)	11.00 (0.97)	2.86 (0.84)	17.41 (0.88)
PINN-semi, Euler residual along the trajectory	13.37 (0.84)	11.31 (1.00)	3.01 (0.89)	17.49 (0.88)
Black-box PINN (PINN4SOH-type dynamics)	14.15 (0.89)	10.91 (0.96)	2.80 (0.83)	17.49 (0.88)
PINN-semi	13.44 (0.84)	10.85 (0.96)	2.58 (0.76)	17.30 (0.87)

still helps at low label fractions (geometric-mean ratio 0.716 at 10 %) but not at 70 % (1.002).

Table 5 adds the sequence models and the tuned hybrid on the same folds. Looking at several cycles helps the neural networks: CNN, LSTM and Transformer reach 0.36–0.93 times the MLP error (significant in 42 of 48 comparisons) and beat PINN-semi in 14 of 16 settings, but they are worse than the GBDT in 40 of 48 and worse than PINN-hybrid in 47 of 48 comparisons. The tuned hybrid is lower than the GBDT significantly in 11 of 16 settings and never significantly higher. Adding causal feature history (exponential moving averages and recent slopes) helps on TJU and HUST from 30 % labels but hurts on XJTU; selecting history per dataset and label fraction on validation (a post hoc analysis, last column) gives a lower MAE than the GBDT in all 16 settings, 13 significantly.

5.4 Revisiting the PINN4SOH benchmark

PINN4SOH [6] is the reference result on these four datasets. We reconstructed its 39 evaluation groups (one model per batch for XJTU and TJU, one per dataset for MIT and HUST with 23 and 20 fixed test cells, and few-shot runs with $K = 1, \ldots, 4$ labelled cells for XJTU and HUST) from the `data path` lines of the training logs released with the code, and evaluate everything as the paper does: MAE per test cell, averaged over cells, runs and groups.

The released configuration. The training logs record, for every epoch, the mean "physics loss" of the monotonicity term. In the first released version of the model this term is `relu(y2-y1).sum()`, computed from the labels of consecutive cycles only. In all 39 groups the logged value is one constant across all epochs of all ten runs, and in all of them it equals this expression evaluated on the training rows that the original data loader builds from our copy of the data

Table 5: Per-cell MAE ($\times 10^{-3}$) of the main models on the same folds and labelled cells (three repetitions $\times$ five folds). PINN-hybrid: weights and filter window tuned on validation. †Hybrid with or without causal feature history, selected on validation per dataset and label fraction (post hoc). Bold: lowest per row.

Dataset	Labels	MLP	CNN	LSTM	Transformer	GBDT	PINN-semi	PINN-hybrid	+ history†
XJTU	10 %	23.27	16.01	16.62	17.63	14.01	14.65	**12.71**	**12.71**
	30 %	15.61	12.68	13.98	13.71	10.87	13.35	**10.78**	**10.78**
	50 %	14.10	11.69	11.98	12.05	10.18	13.20	**10.02**	**10.02**
	70 %	12.48	10.98	10.50	10.99	9.64	12.53	**9.50**	**9.50**
TJU	10 %	15.81	14.22	13.08	14.14	12.60	14.01	**11.70**	**11.70**
	30 %	11.71	8.89	8.69	9.89	8.06	11.39	8.00	**7.78**
	50 %	9.96	7.31	7.24	9.31	7.17	10.74	7.23	**6.92**
	70 %	9.71	7.36	7.09	8.96	6.96	10.80	7.11	**6.82**
MIT	10 %	9.02	3.30	3.26	3.87	3.15	3.29	**2.52**	**2.52**
	30 %	3.39	2.18	2.18	2.77	1.97	2.50	**1.75**	**1.75**
	50 %	3.06	1.99	2.04	2.71	1.74	2.45	**1.59**	**1.59**
	70 %	2.85	1.93	1.99	2.59	1.59	2.38	**1.48**	**1.48**
HUST	10 %	41.62	22.77	24.51	21.95	23.84	22.76	**20.36**	**20.36**
	30 %	20.00	17.42	17.14	17.45	18.73	17.10	15.99	**15.34**
	50 %	18.13	15.04	15.52	16.00	15.82	16.42	14.26	**13.17**
	70 %	17.19	14.19	14.76	14.93	14.60	16.38	13.76	**12.24**

(e.g. 0.043214 for the XJTU 2C batch). The term therefore adds a constant and no gradient: the published numbers come from a model trained with the data and PDE terms only. The match also shows that our reconstructed splits and data reproduce the training rows of every published group. Two later revisions replaced the term, first by `relu(u2-u1).sum()` and then by `relu((u2-u1)*(y1-y2)).sum()`, and the latest commit notes that the loss weights α and β changed after a bug fix; the default weights in the released scripts no longer match those in the logs.

Re-running the original code. We ran the released model and data-loading classes unchanged on the reconstructed splits, for two versions of the model: the version that produced the published numbers, with the logged hyperparameters (10 seeded runs per group, as in the paper), and the current version with its default weights (3 runs per group) (Table 6, Figure 3). The original code needs pandas 1.x semantics; we used pandas 2.3, PyTorch 2 and CPUs, whereas the original runs used GPUs and older libraries. The re-run of the published version gives 1.04–1.19 times the published error on the four full datasets and 0.97–1.20 times in the few-shot settings. Per test cell, the re-run is significantly worse on 2 of the 4 full datasets. The standard deviation over runs is 4–34 % of the mean, so single runs differ by as much as these gaps. Since the training rows are identical, the remaining differences are hardware, library versions and random seeds; we could not reproduce the original environment and do not know which of them causes the systematic part. We therefore report both the published numbers and the re-run, and use the re-run for paired tests because it gives per-cell predictions under known conditions. The corrected current version gives 0.93–1.02 times the error of the published version across the 12 settings.

Same splits, same preprocessing. With the original preprocessing (whole-life 3σ filter on all columns including capacity, per-cell min–max normalisation of features and cycle index), PINN-hybrid in its fixed configuration ($\omega = \frac{1}{2}$, $w = 10$, three seeds, no tuning on these splits) has 0.76–0.92 times the error of the re-run original code on the full datasets (significantly lower in 2 and higher in 0 of 4) and 0.31–0.67 times in the few-shot settings (lower in 5, higher in 0 of 8). Against the published numbers the ratios are 0.84–1.10 and 0.37–0.73. In the few-shot

Table 6: Per-cell MAE ($\times 10^{-3}$) on the PINN4SOH splits, averaged over test cells, runs and groups as in the original paper. Re-runs: the released code as published (label-only monotonicity term, logged hyperparameters; ten runs per group, seeds 0–9) and as currently released (corrected term, new defaults; three runs per group, seeds 0–2); ±: standard deviation over runs. Ours: three seeds. K: labelled training cells in the few-shot experiments. $^*/^\dagger$: hybrid with the original preprocessing significantly lower/higher than the re-run as published (Wilcoxon over test cells, Holm across the 12 rows).

		PINN4SOH			ours		
Dataset	Setting	published	re-run, as published	re-run, current	hybrid, orig. prep.	GBDT, orig. prep.	hybrid, leakage-free
XJTU	all cells	7.85	8.66 ± 1.04	–	6.62	6.82	7.81
TJU	all cells	9.79	11.21 ± 0.47	–	8.78*	8.63	6.16
MIT	all cells	6.18	7.36 ± 0.69	7.48 ± 0.97	6.80	6.15	1.43
HUST	all cells	7.57	7.90 ± 0.80	7.38 ± 0.37	6.45*	6.16	12.51
XJTU	$K = 1$	31.54	30.70 ± 8.47	–	–	–	–
	$K = 2$	10.06	11.54 ± 1.34	–	–	–	–
	$K = 3$	7.98	9.24 ± 1.13	–	–	–	–
	$K = 4$	7.62	9.17 ± 1.09	–	–	–	–
HUST	$K = 1$	39.42	47.13 ± 15.83	–	14.71*	10.08	46.72
	$K = 2$	16.94	18.07 ± 3.15	–	11.53*	9.66	23.82
	$K = 3$	14.59	15.10 ± 1.41	–	9.16*	7.73	21.13
	$K = 4$	13.62	14.71 ± 0.60	–	9.91*	8.58	22.64

settings the unlabelled training cells enter the physics losses of our model, which is where the semi-supervised design pays off. The strongest model under these conditions, however, is not a PINN: a GBDT with the same preprocessing has 0.77–0.84 times the re-run error on the full datasets and 0.21–0.58 times in the few-shot settings, and is significantly lower than PINN-hybrid in 3 of the 12 settings (higher in 0; ratio 0.69–1.03). With per-cell min–max normalisation, the normalised cycle index is the fraction of life already elapsed, a feature that a tree exploits directly.

Without the leaks. With our leakage-free preprocessing the same model has 0.99 (XJTU), 0.63 (TJU), 0.23 (MIT), 1.65 (HUST) times the published error on the full datasets (per test cell significantly lower on 2 and higher on 1 of 4). The direction follows the leakage audit (Section 5.1): the original preprocessing helps on datasets where the elapsed fraction of life determines SOH (HUST) and hurts where the absolute feature level matters (TJU, MIT). A comparison that changes the preprocessing therefore cannot separate the model from the protocol, and the published numbers on the few-shot settings, where knowing the elapsed fraction of life is almost knowing SOH, mostly reflect the normalisation.

5.5 Where to spend capacity measurements

Labelling every cycle of 30 % of the cells uses 43–44 % of the training cycles. Measuring all training cells every $0.05L$ cycles uses 1.2–6.0 %, 7–37 times fewer, yet PINN-hybrid trained this way has a lower MAE than with 30 % of the cells fully labelled on all four datasets (ratio 0.85–0.93, all significant) and 0.97–1.05 times the error with 70 % labelled (Table 7). Consecutive cycles of a cell repeat each other's information, whereas the dominant error, the level offset of each cell, can only be learned from labels of that cell; spreading measurements over cells pays for that. The same holds for every model: a GBDT on interpolated labels at $0.05L$ is within 1.00–1.04 times the GBDT with 70 % of the cells labelled. At the sparsest schedule ($0.5L$, about three measurements per cell), linear interpolation misses the late acceleration, and on XJTU and MIT the GBDT on interpolated labels is up to three times worse than the PINNs, whose monotonicity and kinetics losses connect the measurements.

Updating a cell in service. If the cell in service is also measured every $0.1L$ cycles (1–3 % of its cycles) and the estimate is updated with the Kalman filter, the error on unmeasured cycles

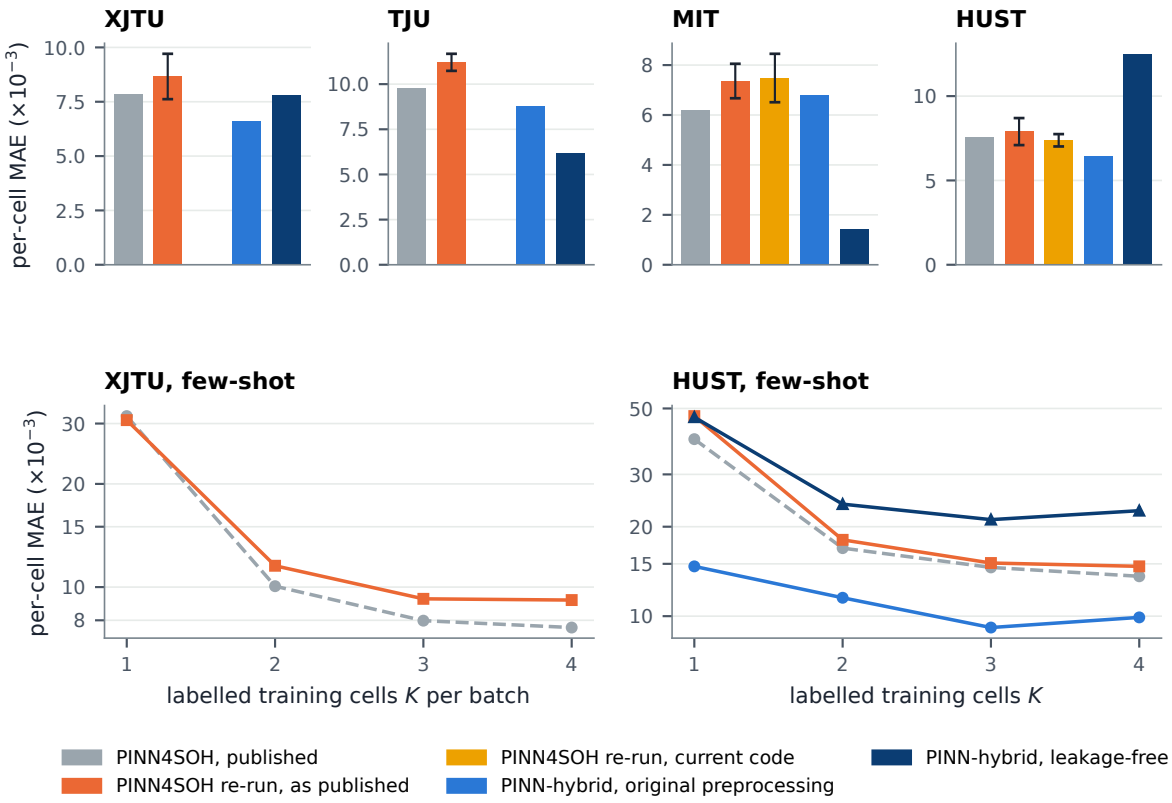


Figure 3: PINN4SOH splits. Top: full training sets, per dataset. Bottom: few-shot experiments with K labelled cells (XJTU per batch, HUST overall). Published: numbers released with PINN4SOH. Re-run: the released code as published (label-only monotonicity term, ten runs per group) and as currently released (corrected term, three runs per group). Ours: PINN-hybrid with the original preprocessing and with the leakage-free preprocessing.

falls to 1.2–7.5 (10^{-3}), 0.34–0.77 times the GBDT trained with 70 % of the cells labelled and not updated; with the same schedule in service, the hybrid trained with 70 % of the cells labelled gives 1.0–7.4, so once the cell is measured the training labels matter little. A single measurement at commissioning helps little, because the capacity of many cells still rises during the first tens of cycles. Across the four datasets, four label fractions and three sparse schedules, the filter beats the uncorrected estimate in 46 of 48 cases and beats simply holding the last measured value in 48 of 48. When every previous cycle is measured, however, holding the last value is better than the filter (ratio of the filter to holding 1.17–1.52): the cycle-to-cycle change predicted by the model is noisier than the true change. The learned update model addresses exactly this: it is lower than the Kalman filter in 47 of 48 sparse settings (ratio 0.15–1.06) and lower than holding the last value in 48 of 48 (0.07–0.53); with every previous cycle measured it beats the filter in 16 of 16 settings and is significantly lower than holding in 12 of 16 (significantly higher in one, XJTU at 10 % labels).

6 Discussion

When does the physics help? Among networks that see one cycle, the semi-supervised PINN should be preferred when labelled cells are few (10–30 %) and many unlabelled cells exist; the gain comes from the label-free losses on those cells, and most of it from the monotonicity constraint along real trajectories. When labels are plentiful, the physics adds little. Trees remain strong on these engineered features, and the most useful role for the PINNs is as a component

Table 7: Measurement budget: per-cell MAE ($\times 10^{-3}$) on test cells, three repetitions; in parentheses, the percentage of training cycles with a capacity measurement (70 % of the cells labelled is all of them). Sparse: every training cell measured at its first cycle, every qL cycles and its last cycle. Last column: the cell in service is also measured every $0.1L$ cycles and the estimate is updated with the Kalman filter; error on unmeasured cycles.

	labels per cell			sparse, all cells		sparse + in service
Dataset	Hybrid, 30 %	Hybrid, 70 %	GBDT, 70 %	Hybrid, $0.1L$	Hybrid, $0.05L$	Hybrid, $0.1L$
XJTU	11.43 (44.1)	10.00	9.64	10.51 (3.2)	10.28 (6.0)	7.46
TJU	8.52 (42.6)	7.76	6.96	8.28 (3.1)	7.54 (5.9)	4.13
MIT	1.79 (42.7)	1.59	1.59	1.91 (1.9)	1.67 (3.6)	1.18
HUST	16.19 (43.1)	13.78	14.60	13.87 (0.6)	13.78 (1.2)	5.00

that disagrees with the tree in a different way. Sequence models reach a similar improvement over the MLP by another route, looking at the recent cycles of the cell, which suggests using a sequence model as the solution network of a PINN.

Leakage changes conclusions, in both directions. Row-wise splits always flatter the model. Per-cell normalisation over the whole life is subtler: it reveals the elapsed fraction of life, which nearly determines SOH when all cells end at similar SOH (XJTU, HUST), but erases the absolute level of the features, which carries information when end-of-life SOH varies (TJU, MIT). Its effect on a reported number can therefore be an improvement or a degradation, and neither reflects deployment, where the life of the cell is unknown.

Lessons from the benchmark re-run. Three practical points follow. Reported numbers should come with per-cell errors and the exact splits, which made the comparison possible here; the logs released with PINN4SOH allowed us to reconstruct both. Released code should be the code that produced the numbers, or the differences should be listed: a constant loss term and changed weights alter the method that is being compared. And preprocessing that uses the whole life of a cell should be avoided, because its effect on the error depends on the dataset and can be large in either direction.

Limitations. (i) Validation cells are fully labelled and lie outside the label budget in all experiments, including the sparse-measurement ones. (ii) Physics weights were set once and not tuned per dataset. (iii) The physical parameters λ and E_a are not identified by these data. (iv) The hybrid combines the models at the prediction level; physics is not part of the tree. (v) Later experiments use three of the five repetitions. (vi) Variants marked as tuned or post hoc were selected on validation data but defined after test results of the fixed versions were known. (vii) Our re-run of PINN4SOH uses CPUs and recent library versions; run-to-run variation is reported, but the original hardware and software could not be reproduced exactly. (viii) All data are laboratory cycling at controlled temperature; field data with irregular measurements remain to be tested.

7 Conclusion

Under a leakage-free protocol on 387 cells, a semi-supervised PINN with label-free physics losses substantially improves a neural network when few cells are labelled, but a tuned GBDT remains the strongest single model; averaging PINNs with the tree and filtering the result causally gives the best estimator in most settings. Common preprocessing choices change reported errors by factors of up to 2.8, in either direction, and a re-run of a published benchmark shows how much released code, logged configurations and evaluation details matter. For deployment, the

useful question is not how many cells to label but where to place capacity measurements: sparse measurements on every cell, and a few measurements of the cell in service, are worth more than full labels on a subset.

Data and code availability

The datasets are public and were obtained from the PINN4SOH repository (commit `cc3cc50`). All code, the reconstructed PINN4SOH splits, the run configurations and the scripts that produce every table and figure of this paper are available in the accompanying repository; `python run.py all` reproduces the experiments, the benchmark re-run and this manuscript.

References

[1] Darius Roman, Saurabh Saxena, Valentin Robu, Michael Pecht, and David Flynn. Machine learning pipeline for battery state-of-health estimation. *Nature Machine Intelligence*, 3: 447–456, 2021.

[2] Xiaosong Hu, Le Xu, Xianke Lin, and Michael Pecht. Battery lifetime prognostics. *Joule*, 4 (2):310–346, 2020.

[3] Valentin Sulzer, Peyman Mohtat, Antti Aitio, et al. The challenge and opportunity of battery lifetime prediction from field data. *Joule*, 5(8):1934–1955, 2021.

[4] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[5] George Em Karniadakis, Ioannis G. Kevrekidis, Lu Lu, Paris Perdikaris, Sifan Wang, and Liu Yang. Physics-informed machine learning. *Nature Reviews Physics*, 3:422–440, 2021.

[6] Fujin Wang, Zhi Zhai, Zhibin Zhao, Yi Di, and Xuefeng Chen. Physics-informed neural network for lithium-ion battery degradation stable modeling and prognosis. *Nature Communications*, 15:4332, 2024.

[7] Pengfei Wen, Zhi-Sheng Ye, Yong Li, Shaowei Chen, Pu Xie, and Shuai Zhao. Physics-informed neural networks for prognostics and health management of lithium-ion batteries. arXiv:2301.00776, 2023.

[8] Sina Navidi, Adam Thelen, Tingkai Li, and Chao Hu. Physics-informed machine learning for battery degradation diagnostics: a comparison of state-of-the-art methods. arXiv:2404.04429, 2024.

[9] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.

[10] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems*, volume 35, 2022.

[11] Kristen A. Severson, Peter M. Attia, Norman Jin, et al. Data-driven prediction of battery cycle life before capacity degradation. *Nature Energy*, 4:383–391, 2019.

[12] Jiangong Zhu, Yixiu Wang, Yuan Huang, et al. Data-driven capacity estimation of commercial lithium-ion batteries from voltage relaxation. *Nature Communications*, 13:2261, 2022.

[13] Robert R. Richardson, Michael A. Osborne, and David A. Howey. Gaussian process regression for forecasting battery state of health. *Journal of Power Sources*, 357:209–219, 2017.

[14] Yongzhi Zhang, Rui Xiong, Hongwen He, and Michael G. Pecht. Long short-term memory recurrent neural network for remaining useful life prediction of lithium-ion batteries. *IEEE Transactions on Vehicular Technology*, 67(7):5695–5705, 2018.

[15] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[16] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. arXiv:1803.01271, 2018.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[18] Marc Doyle, Thomas F. Fuller, and John Newman. Modeling of galvanostatic charge and discharge of the lithium/polymer/insertion cell. *Journal of The Electrochemical Society*, 140 (6):1526–1533, 1993.

[19] Gregory L. Plett. *Battery Management Systems, Volume I: Battery Modeling*. Artech House, 2015.

[20] Muratahan Aykol, Chirranjeevi Balaji Gopal, Abraham Anapolsky, et al. Perspective—combining physics and machine learning to predict battery lifetime. *Journal of The Electrochemical Society*, 168:030525, 2021.

[21] Jiahuan Lu, Rui Xiong, Jinpeng Tian, Chenxu Wang, and Fengchun Sun. Deep learning to estimate lithium-ion battery state of health without additional degradation experiments. *Nature Communications*, 14:2760, 2023.

[22] Guijun Ma, Songpei Xu, Benben Jiang, et al. Real-time personalized health status prediction of lithium-ion batteries using deep transfer learning. *Energy & Environmental Science*, 15: 4083–4094, 2022.

[23] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6): 80–83, 1945.

[24] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[25] Claude Nadeau and Yoshua Bengio. Inference for the generalization error. *Machine Learning*, 52:239–281, 2003.

[26] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[27] Jerome H. Friedman. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232, 2001.

[28] Guolin Ke, Qi Meng, Thomas Finley, et al. LightGBM: a highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[29] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.